



复旦微电子

FM15L013 安全控制器

技术手册

2021.11



本资料是为了让用户根据用途选择合适的上海复旦微电子集团股份有限公司（以下简称复旦微电子）的产品而提供的参考资料，不保证本资料中不含任何瑕疵。

本资料不转让属于复旦微电子或者第三者所有的知识产权以及其他权利的许可。

在使用本资料所记载的信息最终做出有关信息和产品是否适用的判断前，请您务必将所有信息作为一个整体系统来进行评价。

采购方对于选择与使用本文描述的复旦微电子的产品和服务全权负责，复旦微电子不承担采购方选择与使用本文描述的产品和服务的责任。除非以书面形式明确地认可，复旦微电子的产品不推荐、不授权、不担保用于包括军事、航空、航天、救生及生命维持系统在内的，由于失效或故障可能导致人身伤亡、严重的财产或环境损失的产品或系统中。

未经复旦微电子的许可，不得翻印或者复制全部或部分本资料的内容。

今后日常的产品更新会在适当的时候发布，恕不另行通知。在购买本资料所记载的产品时，请预先向复旦微电子在当地的销售办事处确认最新信息，并请您通过各种方式关注复旦微电子公布的信息，包括复旦微电子的网站(<http://www.fmsh.com/>)。

如果您需要了解有关本资料所记载的信息或产品的详情，请与上海复旦微电子集团股份有限公司在当地的销售办事处联系。

商标

上海复旦微电子集团股份有限公司的公司名称、徽标以及“复旦”徽标均为上海复旦微电子集团股份有限公司及其分公司在中国的商标或注册商标。

上海复旦微电子集团股份有限公司在中国发布，版权所有。

章节列表

章节列表	3
1 产品综述	5
1.1 介绍	5
1.2 功能框图	5
1.3 管脚封装	5
1.3.1 SOP8 封装定义	5
2 存储器配置	6
2.1 存储器访问地址	6
2.2 访问权限	6
3 寄存器	7
3.1 介绍	7
3.2 寄存器位权限描述	7
3.3 外部寄存器描述	7
3.3.1 Flash_intf 模块	7
3.3.2 PMU 模块	11
3.3.3 RCC 模块	12
3.3.4 PAD 模块	14
3.3.5 TIMER 模块	22
3.3.6 WDT 模块	24
3.3.7 CRC 模块	25
3.3.8 INTERRUPT 模块	27
4 异常和中断	30
4.1 概述	30
4.2 异常	30
4.2.1 复位	30
4.2.2 NMI	30
4.2.3 硬件错误	31
4.2.4 SysTick	31
4.3 中断	31
4.3.1 管脚电平中断	31
4.3.2 计数器 1 计数中断	31
4.3.3 计数器 2 计数中断	31
4.3.4 数据 NVM 校验错误报警中断	31
4.3.5 软中断	31
5 NVM 擦写流程	33
5.1 概述	33
6 低功耗流程	34
6.1 概述	34
6.2 操作流程	34
7 CRC 计算	35
7.1 概述	35
7.2 操作流程	35
8 时钟控制	36
8.1 概述	36



8.2	STRETCH 的使用.....	36
9	计时器.....	37
9.1	概述.....	37
9.2	24-BITS 定时器	37
9.2.1	简介.....	37
9.2.2	寄存器定义.....	37
9.2.3	操作流程.....	38
9.3	独立计时器.....	38
9.3.1	概述.....	38
9.3.2	计时器精度与计时范围.....	39
9.3.3	操作流程.....	39
10	WDT	41
	上海复旦微电子集团股份有限公司销售及服	43

1 产品综述

1.1 介绍

FM15L013 为一款安全主控芯片，能满足物联网、智能家电等多种应用领域的的安全控制需求。

- 接口
 - 6 个 GPIO
- 存储器
 - NVM 空间：132KB
 - RAM 空间：8KB

1.2 功能框图

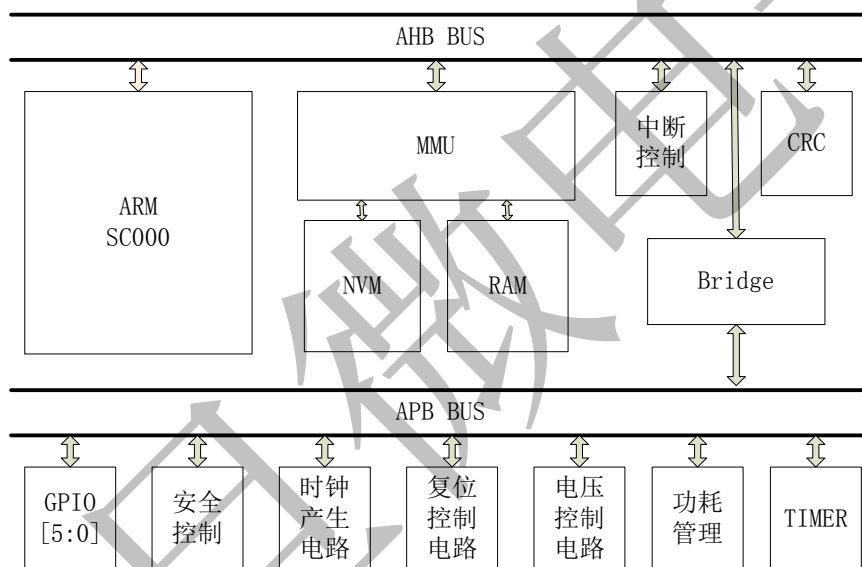


图 1-1: FM15L013 安全控制器功能框图

1.3 管脚封装

1.3.1 SOP8 封装定义

封装序号	引脚名称	类型	说明
1	GPIO3	输入输出	双向数据信号
2	GPIO5	输入输出	双向数据信号
3	GPIO4	输入输出	双向数据信号
4	GND	电源	地
5	GPIO2	输入输出	双向数据信号
6	GPIO0	输入输出	双向数据信号
7	GPIO1	输入输出	双向数据信号
8	VCC	电源	电源

表 1-1: SOP8 管脚定义

2 存储器配置

2.1 存储器访问地址

FM15L013 所有存储器使用大端模式存储,用户访问存储器的逻辑段及地址范围定义如下:

逻辑地址范围	逻辑区域名	说明
000X_XXXXh	用户程序区	通用程序区,存放用户程序。 非挥发性存储器,掉电后数据保持。
0080_XXXXh	CPU 缓存	数据缓存区,存放程序的缓存变量。
00BX_XXXXh	用户通用数据区	用户的数据主存储区,存储用户数据。
00DE_0XXXh	产品信息区	片制造商信息、产品信息等内容。
00EX_XXXXh	外设寄存器区	各个功能模块的控制和状态寄存器。
E000_XXXXh	CPU 内部寄存器	CPU 内部寄存器。

表格 2-1: FM15L013 存储器逻辑地址映射表

2.2 访问权限

FM15L013 存储器访问权限如下表所示:

逻辑区域名	访问权限	说明
用户程序区	R/W/X	通用程序区,存放用户程序。 非挥发性存储器,掉电后数据保持。
CPU 缓存	R/W	数据缓存区,可读可写。
用户通用数据区	R/W	用户的数据主存储区,可读可写。
产品信息区	R	芯片产品信息区域,用户只读。
外设寄存器区	R/W	各个功能模块的控制和状态寄存器,可读可写。
CPU 内部寄存器	R/W	CPU 内部寄存器,可读可写。

表格 2-2 FM15L013 存储器访问权限表

3 寄存器

3.1 介绍

FM15L013 的寄存器包括 CPU 的内部寄存器（SFR）和寄存器区的外部寄存器。

CPU 的内部寄存器只能由 CPU 进行访问，负责 CPU 的正常工作。

外部寄存器是各模块与 CPU 之间进行控制和通信的接口。CPU 和内部控制电路都有相应的访问权。

所有寄存器都以 WORD 为单位进行地址分配，WORD 地址范围内按照大端模式存储数据。

寄存器的有效地址范围是 0x00E0_0000~0x00EF_FFFF。

对外部功能模块寄存器的更改操作必须要保证寄存器中 RFU 位数据的正确性，寄存器的写操作流程如下：

- 1) 先读出原始寄存器中的值
- 2) 对数据做处理，只更改目标需要修改的位数值，保留寄存器其他位信息不变
- 3) 将处理好的数据以 word 为单位写入目标寄存器中

3.2 寄存器位权限描述

每一个寄存器里的每一位按其功能都有不同的读写权限

缩写	权限	描述
D	动态	这些位可以被内部控制电路自动改写
—	不可读写	读操作返回全 0，写操作无法写入数据。
R	可读	这些位可以被读出，写操作无法写入数据
W	可写	这些位可以被写入。
W ₁	写 1	只可以写 1，写 0 操作将无法写入。
W ₀	写 0	只可以写 0，写 1 操作将无法写入。

表 3-1: 寄存器位的权限及描述

3.3 外部寄存器描述

3.3.1 Flash_intf 模块

3.3.1.1 FLASH_EW_START_STEP1

寄存器名	FLASH_EW_START_STEP1
功能	NVM 操作启动寄存器
地址	0x00E0_0000
复位值	0x00000000

位描述

位	位名	权限	功能描述
---	----	----	------



31~4	RFU	-	-
3~0	nvm_op_start	RW	NVM 操作启动寄存器。逻辑上 32bit，实际物理上 4bit 寄存器，只锁存低 4bit 值。 向寄存器写入 32'h55AA_AA55 以启动擦写流程。 ● 只有配置了该寄存器后，才可以配置 flash_op_mode 寄存器。 ● 该寄存器配置非 32'h55AA_AA55 值无效。 ● 配置了 flash_op_mode 寄存器后，如果写本寄存器，会复位操作流程，需重新配置 flash_op_mode 寄存器。 ● 该寄存器和 flash_ew_start_step2 寄存器物理上为一个寄存器，只是读写地址不同。 ● 写了 flash_ew_start_step1 后，读出为 4'h5， ● 写了 flash_ew_start_step2 后，读出值为 4'hA。

3.3.1.2 FLASH_OP_MODE

寄存器名	FLASH_OP_MODE
功能	NVM 擦写操作配置寄存器
地址	0x00E0_0004
复位值	0x00000000

位描述

位	位名	权限	功能描述
31~10	RFU	-	-
9~8	nvm_cell_0_program_en	RW	明文写全 0 模式使能位： 01: 明文写全 0 模式使能 00/10/11: 明文写全 0 模式不使能
7~6	nvm_chip_erase_en	RW	chip 全擦模式使能位，sector 擦除使能后，此位无效： 01: chip 全擦模式使能 00/10/11: chip 全擦模式不使能
5~4	nvm_sector_erase_en	RW	sector 擦除模式使能位： 01: sector 擦除模式使能 00/10/11: sector 擦除模式不使能
3~2	nvm_page_erase_en	RW	page 擦除模式使能位： 01: page 擦除模式使能 00/10/11: page 擦除模式不使能
1~0	nvm_program_en	RW	编程 program 模式使能位： 01: 编程 program 模式使能 00/10/11: 编程 program 模式不使能

注：1. program、page erase、sector erase、chip erase 只能使能一种操作，硬件判断 7~0 一共 8bit 决定操作模式。2. 对 Flash_ew_start 寄存器写 step1 的值或其他非 step1、step2 的值，会清零该寄存器的值。

3.3.1.3 FLASH_EW_START_STEP2

寄存器名	FLASH_EW_START_STEP2
------	----------------------



功能	NVM 操作启动寄存器
地址	0x00E0_0008
复位值	0x00000000

位描述

位	位名	权限	功能描述
31~0	nvm_op_start	RW	NVM 操作启动寄存器。可以通过向寄存器写入 32'h5AA5_A55A 以启动擦写流程。 ● 只有配置了flash_op_mode寄存器后, 才可以配置该寄存器。 ● 该寄存器配置非32'h5AA5_A55A值无效。 ● 该寄存器和flash_ew_start_step1寄存器物理上为一个寄存器, 只是读写地址不同。 ● 写了flash_ew_start_step1后, 读出为4'h5。 ● 写了flash_ew_start_step2后, 读出值为4'hA。

3.3.1.4 FLASH_INTF_CFG

寄存器名	FLASH_INTF_CFG
功能	Flash_intf 功能配置寄存器
地址	0x00E0_000C
复位值	0x00000001

位描述

位	位名	权限	功能描述
31~10	RFU	-	-
9-8	flash_cell_chk_en	RW	明文校验使能位: 01: 表示使能; 其他: 表示禁止。
7-6	RFU	-	-
5-4	stretch_vlaue	RW	读取 Flash 的 stretch: 2'b00: 无 stretch 2'b01: 加 1 个 stretch 2'b10: 加 2 个 stretch 2'b11: 加 3 个 stretch
3~2	RFU		
1~0	double_read_en	RW	数据 Flash 安全读操作使能位, 该位使能后, 每个 Flash 的读周期会变成两个读周期, 用来做 Flash 的数据读校验, 增加 Flash 的访问安全。 01: 标准的读模式 00/10/11: double read 读模式

3.3.1.5 FLASH_INTF_STATUS

寄存器名	FLASH_INTF_STATUS
功能	Flash 接口状态寄存器



地址	0x00E0_0010
复位值	0x00000000

位描述

位	位名	权限	功能描述
31~6	RFU	-	-
5-4	flash_cell_chk_res	RDW	明文校验结果，硬件置位，软件清零 00/11: 当前读地址的物理 cell 非全 0 或全 1 01: 当前读地址的物理 cell 为全 0 10: 当前读地址的物理 cell 为全 1
3-2	RFU	-	-
1	data_flash_chk_err_flag	RDW	数据 Flash 校验错误标识。产生中断。 1—数据 Flash 校验出错。 硬件置位，软件写 1 清中断。（写零无效，忽略操作）
0	flash_ew_over_flag	RDW	Flash 擦写操作完成标识，硬件置位，软件写 1 清零。 将产生 Flash_ew_over 的中断。（写零无效，忽略操作） 0—Flash 擦写未完成； 1—Flash 擦写操作完成；

3.3.1.6 FLASH_CELL_CHK_ADDR

寄存器名	FLASH_CELL_CHK_ADDR
功能	明文校验地址配置寄存器
地址	0x00E0_0014
复位值	0x00000000

位描述

位	位名	权限	功能描述
31~24	RFU	-	-
23-9	flash_cell_chk_res	RW	明文校验 page 地址设定。 在设定了明文校验使能后，当读取该配置的 page 内地址时，将进行明文校验，检查是否为全 0 和全 1。
8-0		-	

3.3.1.7 USER_FLAG

寄存器名	USER_FLAG
功能	用户标志寄存器
地址	0x00E0_0020
复位值	0x00000000

位描述

位	位名	权限	功能描述
31~16	RFU	-	-
15-0	user_flag	RW	用作用户自定义的标志记录。

3.3.2 PMU 模块

3.3.2.1 PMU_CR0

寄存器名	PMU_CR0
功能	功耗模式控制寄存器 0
地址	0x00E6_0100
复位值	0x00008812

位描述

31: 0	低功耗配置	WR	需固定写入 0x00008831。
-------	-------	----	-------------------

3.3.2.2 PMU_USER_FR

寄存器名	PMU_USER_FR
功能	用户标志寄存器
地址	0x00E6_0104
复位值	0x00000000

位描述

位	位名	权限	功能描述
31:16	RFU	-	保留位，读数为 0。
15:0	pmu_user_flag[15:0]	WR	低功耗下用户标志寄存器，仅供软件使用。 复位值：16'h0000。

3.3.2.3 PMU_INTER

寄存器名	PMU_INTER
功能	中断使能寄存器
地址	0x00E6_0108
复位值	0x00000000

位描述

位	位名	权限	功能描述
31:1	RFU	-	保留位，读数为 0。
0	inte_slp_wkup	WR	SLEEP 模式唤醒中断使能，高有效。 0: SLEEP 模式唤醒后，不产生中断标志（复位值）。 1: SLEEP 模式唤醒后，产生中断标志。

3.3.2.4 PMU_INTFR

寄存器名	PMU_INTFR
功能	中断使能寄存器
地址	0x00E6_010C
复位值	0x00000000

位描述

位	位名	权限	功能描述
31:1	RFU	-	保留位，读出为 0。
0	inte_slp_wkup	WR	SLEEP 模式唤醒中断标志，高有效。 pmu_inter[inte_slp_wkup]使能后，当芯片从 SLEEP 模式唤醒时，由硬件自动置 1。 写 1 清 0，对该寄存器写 0 无效。 0：芯片未从 SLEEP 模式被唤醒（复位值）。 1：芯片从 SLEEP 模式被唤醒。

3.3.2.5 PMU_CR1

寄存器名	PMU_CR1
功能	功耗模式控制寄存器 1
地址	0x00E6_0110
复位值	0x00000000

位描述

位	位名	权限	功能描述
31:4	RFU	-	保留位，读出为 0。
3:0	pmode[3:0]	WR	功耗模式寄存器。 4'h5：芯片进入 SLEEP 模式。 4'hA：芯片进入 STOP 模式。 其它：芯片保持在 ACTIVE 模式（复位值）。

3.3.3 RCC 模块

3.3.3.1 SYSCLK_CFG

寄存器名	SYSCLK_CFG
功能	系统时钟配置寄存器
地址	0x00E6_0200
复位值	0x00000200

位描述

位	位名	权限	功能描述
31~16	RFU	-	-
15	mcg_clk_sel	RW	0：mcg_clk 选择 48MHz 时钟源 1：mcg_clk 选择 32MHz 时钟源
14~5	RFU	-	-
4~0	cpu_n_cfg[4:0]	RW	固定频率工作模式下，Cpu 工作时 mcg_clk 频率分频参数配置，1~32 分频

3.3.3.2 PERICKL_CFG0

寄存器名	PERICKL_CFG0
功能	模块时钟控制寄存器
地址	0x00E8_0100

复位值	0x0003_311B
-----	-------------

位描述

位	位名	权限	功能描述
31-11	RFU	-	-
10	crc_clk_en	RW	CRC 模块时钟使能, 高有效
9	timer_clk_en	RW	timer 模块时钟使能, 高有效
7-0	RFU	RW	固定 8'b00

3.3.3.3 PERICK_CFG1

寄存器名	PERICK_CFG1
功能	模块时钟控制寄存器
地址	0x00E8_0104
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-10	RFU	-	-
9-8	timer_sysclk_div	RW	timer 计数时钟 system_div_clk 分频数 00: 4 01: 8 10: 16 11: 32
7-6	RFU	-	-
5-4	Timer2_clk_sel	RW	Timer2 时钟选择信号 01: system_div_clk 其他: 无
3-2	RFU	-	-
1-0	timer1_clk_sel	RW	Timer1 时钟选择信号 01: system_div_clk 其他: 无

3.3.3.4 RESET_CTRL

寄存器名	reset_ctrl
功能	复位控制寄存器
地址	0x00E6_0204
复位值	0x0000_2000

位描述

位	位名	权限	功能描述
31~14	RFU	—	—
13	pad_nrst_en	RW	引脚复位使能信号, 高有效, 软件清零
12	lockup_rst_en	RW	CPU lockup 复位使能信号, 高有效, 软件清零
11-2	RFU	RW	固定 10'b00
1	cpu_soft_rst	RWD	软件复位 CPU 控制, 高有效



			软件置 1，硬件清零，产生 4 系统时钟周期 CPU 复位脉冲
0	chip_soft_rst	RWD	软件全局复位请求，高有效 置位该位将引起数字电路复位。软件置 1，全局复位自动清零，产生 4 系统时钟周期的复位脉冲

3.3.3.5 RESET_STATUS

寄存器名	reset_status
功能	复位控制寄存器
地址	0x00E6_0208
复位值	0x0000_0001

位描述

位	位名	权限	功能描述
31~9	RFU	—	—
8	cpu_soft_rst_flag	RD	CPU 软件复位标志，高有效，硬件置位
7	RFU	-	-
6	Pad_nrst_flag	RD	引脚复位标志，高有效，硬件置位
5	lockup_rst_flag	RD	CPU lockup 复位标志，高有效，硬件置位
4	cpu_req_rst_flag	RD	CPU 请求系统复位标志，高有效，硬件置位
3	dig_soft_rst_flag	RD	软件复位标志，高有效，硬件置位
2	security_rst_flag	RD	安全复位标志，高有效，硬件置位
1	wdt_rst_flag	RD	看门狗复位标志，高有效，硬件置位
0	pwr_rst_flag	RD	上电复位标志，高有效，硬件置位

3.3.3.6 DEBUG_CTRL

寄存器名	debug_ctrl
功能	调试控制寄存器
地址	0x00E8_0108
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~3	RFU	—	—
2	dbg_timer2_stop_en	RW	CPU debug 时暂停 timer2 模块使能，高有效
1	dbg_timer1_stop_en	RW	CPU debug 时暂停 timer1 模块使能，高有效
0	dbg_wdt_stop_en	RW	CPU debug 时暂停 wdt 模块使能，高有效

3.3.4 PAD 模块

3.3.4.1 GPIO_PAD_SELO

寄存器名	GPIO_PAD_SELO
功能	PAD 选择寄存器 0
地址	0x00E6_0300

复位值	0x0000_0000
-----	-------------

位描述

位	位名	权限	功能描述
31~16	RFU	—	—
15~12	gpio3_pad_sel	RW	PAD 管脚接口选择。选择管脚接口必须放在对应 PAD 的配置修改（GPIO_CFG、GPIO_INT_CFG、GPIO_RSM_CFG）之后进行，即需要先确定 PAD 状态，然后才接通路，否则可能会导致错误状态传入通讯接口。
11~8	gpio2_pad_sel	RW	
7~4	gpio1_pad_sel	RW	
3~0	gpio0_pad_sel	RW	
			0x0 : gpio 0xF : reset_n

注：gpio0 和 gpio1 的通路选择在 SWD 接口启用条件下无论配置何值，都默认走 SWD 通道。

3.3.4.2 GPIO_PAD_SEL1

寄存器名	GPIO_PAD_SEL1
功能	PAD 选择寄存器 1
地址	0x00E6_0304
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~18	RFU	—	—
7~4	gpio5_pad_sel	RW	PAD 管脚接口选择。选择管脚接口必须放在对应 PAD 的配置修改（PAD_CFG、PAD_DLY_FLTR、PAD_INT_CFG、PAD_RSM_CFG）之后进行，即需要先确定 PAD 状态，然后才接通路，否则可能会导致错误状态传入通讯接口。
3~0	gpio4_pad_sel	RW	
			0x0 : gpio 0xF : reset_n

3.3.4.3 GPIO_DOUT0

寄存器名	GPIO_DOUT0
功能	GPIO 输出寄存器 0
地址	0x00E6_0308
复位值	0x0000_1111

位描述

位	位名	权限	功能描述
31~13	RFU	—	—
12	gpio3_dout	RW	gpio3 接口输出信号
11~9	RFU	—	—
8	gpio2_dout	RW	gpio2 接口输出信号
7~5	RFU	—	—

4	gpio1_dout	RW	gpio1 接口输出信号
3~1	RFU	—	—
0	gpio0_dout	RW	gpio0 接口输出信号

3.3.4.4 GPIO_DOUT1

寄存器名	GPIO_DOUT1
功能	GPIO 输出寄存器 1
地址	0x00E6_030C
复位值	0x0000_0111

位描述

位	位名	权限	功能描述
31~5	RFU	—	—
4	gpio5_dout	RW	gpio5 接口输出信号
3~1	RFU	—	—
0	gpio4_dout	RW	gpio4 接口输出信号

3.3.4.5 GPIO_DIN0

寄存器名	GPIO_DIN0
功能	GPIO 输入寄存器 0
地址	0x00E6_0310
复位值	—

位描述

位	位名	权限	功能描述
31~13	RFU	—	—
12	gpio3_din	R	gpio3 管脚输入信号
11~9	RFU	—	—
8	gpio2_din	R	gpio2 管脚输入信号
7~5	RFU	—	—
4	gpio1_din	R	gpio1 管脚输入信号
3~1	RFU	—	—
0	gpio0_din	R	gpio0 管脚输入信号

3.3.4.6 GPIO_DIN1

寄存器名	GPIO_DIN1
功能	GPIO 输入寄存器 1
地址	0x00E6_0314
复位值	—

位描述

位	位名	权限	功能描述
31~5	RFU	—	—
4	gpio5_din	R	gpio5 管脚输入信号
3~1	RFU	—	—
0	gpio4_din	R	gpio4 管脚输入信号

3.3.4.7 GPIO_CFG0

寄存器名	GPIO_CFG0
功能	GPIO 管脚控制配置寄存器
地址	0x00E6_0318
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~16	RFU	—	—
15	gpio1_ana_idly_en	RW	同 GPIO0
14	gpio1_os_en	RW	
13	gpio1_str_pu_en	RW	
12	gpio1_pu_en	RW	
11	gpio1_pd_en	RW	
10	gpio1_ie	RW	
9~8	gpio1_oe	RW	模拟电路延时使能，高有效 one-shot 功能使能，高有效 使能后 one-shot 的脉冲时间为约 60 到 100ns。 强上拉电阻使能，高有效 上拉电阻使能，高有效 下拉电阻使能，高有效 输入使能，高有效
7	gpio0_ana_idly_en	RW	
6	gpio0_os_en	RW	
5	gpio0_str_pu_en	RW	
4	gpio0_pu_en	RW	
3	gpio0_pd_en	RW	
2	gpio0_ie	RW	输出使能 00: 输出关闭 01: 开漏输出 10/11: 驱动输出
1~0	gpio0_oe	RW	

注：在 SWD 接口启用时，由于 SWD 占用 gpio0 和 gpio1，此寄存器除了 gpio0_ana_idly_en 和 gpio1_ana_idly_en 外，其余控制位配置任何值均无效。

3.3.4.8 GPIO_CFG1

寄存器名	GPIO_CFG1
功能	GPIO 管脚控制配置寄存器 1
地址	0x00E6_031C
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~16	RFU	—	—
15	gpio3_ana_idly_en	RW	同 GPIO0
14	gpio3_os_en	RW	
13	gpio3_str_pu_en	RW	
12	gpio3_pu_en	RW	
11	gpio3_pd_en	RW	
10	gpio3_ie	RW	
9~8	gpio3_oe	RW	
7	gpio2_ana_idly_en	RW	同 GPIO0
6	gpio2_os_en	RW	
5	gpio2_str_pu_en	RW	
4	gpio2_pu_en	RW	
3	gpio2_pd_en	RW	
2	gpio2_ie	RW	
1~0	gpio2_oe	RW	

3.3.4.9 GPIO_CFG2

寄存器名	GPIO_CFG2
功能	GPIO 管脚控制配置寄存器 2
地址	0x00E6_0320
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~16	RFU	—	—
15	gpio5_ana_idly_en	RW	同 GPIO0
14	gpio5_os_en	RW	
13	gpio5_str_pu_en	RW	
12	gpio5_pu_en	RW	
11	gpio5_pd_en	RW	
10	gpio5_ie	RW	
9~8	gpio5_oe	RW	
7	gpio4_ana_idly_en	RW	同 GPIO0
6	gpio4_os_en	RW	
5	gpio4_str_pu_en	RW	
4	gpio4_pu_en	RW	
3	gpio4_pd_en	RW	
2	gpio4_ie	RW	
1~0	gpio4_oe	RW	

3.3.4.10 GPIO_INT_CFG0

寄存器名	GPIO_INT_CFG0
功能	GPIO 中断配置寄存器 0

地址	0x00E6_032C
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~14	RFU	—	—
13~12	gpio3_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电平, 防止开启后即触发中断。 00/11: 不触发中断, 即使中断标志已经置起 01 : 高电平触发中断 10 : 低电平触发中断
11~10	RFU	—	—
9~8	gpio2_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电平, 防止开启后即触发中断。 00/11: 不触发中断, 即使中断标志已经置起 01 : 高电平触发中断 10 : 低电平触发中断
7~6	RFU	—	—
5~4	gpio1_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电平, 防止开启后即触发中断。 00/11: 不触发中断, 即使中断标志已经置起 01 : 高电平触发中断 10 : 低电平触发中断
3~2	RFU	—	—
1~0	gpio0_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电平, 防止开启后即触发中断。 00/11: 不触发中断, 即使中断标志已经置起 01 : 高电平触发中断 10 : 低电平触发中断

3.3.4.11 GPIO_INT_CFG1

寄存器名	GPIO_INT_CFG1
功能	GPIO 中断配置寄存器 1
地址	0x00E6_0330
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~6	RFU	—	—
5~4	gpio5_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电平, 防止开启后即触发中断。 00/11: 不触发中断, 即使中断标志已经置起 01 : 高电平触发中断 10 : 低电平触发中断
3~2	RFU	—	—
1~0	gpio4_int_sel	RW	中断触发电平选择, 启用中断前请先确认好 GPIO 默认电



			平，防止开启后即触发中断。 00/11：不触发中断，即使中断标志已经置起 01：高电平触发中断 10：低电平触发中断
--	--	--	---

3.3.4.12 GPIO_INT_STATUS0

寄存器名	GPIO_INT_STATUS0
功能	GPIO 中断状态寄存器 0
地址	0x00E6_0334
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~13	RFU	—	—
12	gpio3_intf	RWD	gpio3 中断标志，写 1 清零，写 0 无效。
11~9	RFU	—	—
8	gpio2_intf	RWD	gpio2 中断标志，写 1 清零，写 0 无效。
7~5	RFU	—	—
4	gpio1_intf	RWD	gpio1 中断标志，写 1 清零，写 0 无效。
3~1	RFU	—	—
0	gpio0_intf	RWD	gpio0 中断标志，写 1 清零，写 0 无效。

注意：若触发中断的 GPIO 电平不变，清零后会立即再次触发中断，在此情况下，清零同时需要先关闭中断。

3.3.4.13 GPIO_INT_STATUS1

寄存器名	GPIO_INT_STATUS1
功能	GPIO 中断状态寄存器 1
地址	0x00E6_0338
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~5	RFU	—	—
4	gpio5_intf	RWD	gpio5 中断标志，写 1 清零，写 0 无效。
3~1	RFU	—	—
0	gpio4_intf	RWD	gpio4 中断标志，写 1 清零，写 0 无效。

注意：若触发中断的 GPIO 电平不变，清零后会立即再次触发中断，在此情况下，清零同时需要先关闭中断。

3.3.4.14 GPIO_RSM_CFG0

寄存器名	GPIO_RSM_CFG0
------	---------------



功能	GPIO 唤醒配置寄存器 0
地址	0x00E6_033C
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~14	RFU	—	—
13~12	gpio3_rsm_sel	RW	gpio3 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒
11~10	RFU	—	—
9~8	gpio2_rsm_sel	RW	gpio2 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒
7~6	RFU	—	—
5~4	gpio1_rsm_sel	RW	gpio1 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒
3~2	RFU	—	—
1~0	gpio0_rsm_sel	RW	gpio0 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒

3.3.4.15 GPIO_RSM_CFG1

寄存器名	GPIO_RSM_CFG1
功能	GPIO 唤醒配置寄存器 1
地址	0x00E6_0340
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~10	RFU	—	—
5~4	gpio5_rsm_sel	RW	gpio5 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒
3~2	RFU	—	—
1~0	gpio4_rsm_sel	RW	gpio4 唤醒触发电平选择 00/11: 不触发唤醒 01 : 高电平触发唤醒 10 : 低电平触发唤醒

3.3.5 TIMER 模块

3.3.5.1 timer_cfg

寄存器名	TIMER_CFG
功能	timer 模块功能配置
地址	0x00E8_0500
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~4	RFU	-	-
3	timer2_reload_en	RW	timer2 reload 使能 1: 计数到设定值后自动重新计数。 0: 计数器到达设定值后, 停止计数, 将 timer_start 寄存器位置 0
2	timer2_int_en	RW	timer2 中断使能开关, 开启后会产生 timer2_int 中断 1: 计数到设定值后产生中断。 0: 计数到设定值后不产生中断。
1	timer1_reload_en	RW	timer1 reload 使能 1: 计数到设定值后自动重新计数。 0: 计数器到达设定值后, 停止计数, 将 timer_start 寄存器位置 0
0	timer1_int_en	RW	timer1 中断使能开关, 开启后会产生 timer1_int 中断 1: 计数到设定值后产生中断。 0: 计数到设定值后不产生中断。

3.3.5.2 timer1_ctrl

寄存器名	TIMER1_CTRL
功能	timer1 使能寄存器
地址	0x00E8_0504
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-1	RFU	-	-
0	timer1_start	RW	timer1 使能寄存器 1: 启动 timer1 计数, 如果没有配置 reload, 计数完成后自动清零。 0: 停止 timer1 计数

注: timer_start 每次启动都会将 timer 的累加计数器当前值 TIMER_VALUE 清零, 重新计数。

3.3.5.3 timer1_data

寄存器名	TIMER1_DATA
------	-------------

功能	timer1 设定值
地址	0x00E8_0508
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-16	RFU	-	-
15~0	counter1_data	RW	计数器 1 设定值，计数器 1 的 counter1_value 达到这个计数值的时候即完成一次计时。

注：写此寄存器值都会将当前计数值 TIMER1_VALUE 清零重新开始一轮计时。

3.3.5.4 timer1_value

寄存器名	TIMER1_VALUE
功能	timer1 计数值
地址	0x00E8_050C
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-16	RFU	-	-
15-0	counter1_value	RWD ₀	计数器 1 的计数值，这个值在 timer1_start 有效后从 0 开始增加，增加到等于 counter1_data 后就完成一次计时。

注：此寄存器写任何值都会将此当前计数值 TIMER1_VALUE 清零重新开始一轮计时。

3.3.5.5 timer2_ctrl

寄存器名	TIMER2_CTRL
功能	timer2 使能寄存器
地址	0x00E8_0510
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-1	RFU	-	-
0	timer2_start	RW	timer2 使能寄存器 1: 启动 timer2 计数，如果没有配置 reload，计数完成后自动清零。 0: 停止 timer2 计数

注：timer_start 每次启动都会将 timer 的累加计数器当前值 TIMER_VALUE 清零，重新计数。

3.3.5.6 timer2_data

寄存器名	TIMER2_DATA
功能	timer2 设定值
地址	0x00E8_0514

复位值	0x0000_0000
-----	-------------

位描述

位	位名	权限	功能描述
31-16	RFU	-	-
15~0	counter2_data	RW	计数器 2 设定值，计数器 2 的 counter2_value 达到这个计数值的时候即完成一次计时。

注：写此寄存器值都会将当前计数值 TIMER2_VALUE 清零重新开始一轮计时。

3.3.5.7 timer2_value

寄存器名	TIMER2_VALUE
功能	timer2 计数值
地址	0x00E8_0518
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31-0	counter2_value	RWD ₀	计数器 2 的计数值，这个值在 timer2_start 有效后从 0 开始增加，增加到等于 counter2_data 后就完成一次计时。

注：此寄存器写任何值都会将此当前计数值 TIMER2_VALUE 清零重新开始一轮计时。

3.3.6 WDT 模块

3.3.6.1 WDT_TIMER_CONFIG

名字	WDT_TIMER_CONFIG
功能	WDT 计数量程配置
地址	0x00E8_0600
复位值	0x03FF_FFFF

位描述

位	位名	权限	功能
31-26			RFU
25-0	wdt_timer_config	RW	Watch Dog 计数量程，当计数器计到此数值时产生硬件复位信号

3.3.6.2 WDT_CLR

名字	WDT_CLR
功能	WDT 清零寄存器
地址	0x00E8_0604
复位值	0x0000_0000

位描述

位	位名	权限	功能
31-8	RFU		-

7-0	wdt_clr	Wc	Watch Dog 清零, 读出为 8'h0, 可写入任意值, 但当且仅当写入 5'h5A 时复位 Watch Dog 的计数器
-----	---------	----	--

3.3.6.3 WDT_EN

名字	WDT_EN		
功能	WDT 使能寄存器		
地址	0x00E8_0608		
复位值	0x0000_0002		

位描述

位	位名	权限	功能
31-2	RFU		-
1-0	wdt_en	RW	Watch Dog 使能, 复位为 10, 写 01 使能 Watch Dog, 一旦使能无法修改, Watch Dog 时钟打开。

3.3.6.4 WDT_CNT

名字	WDT_CNT		
功能	WDT 计数器寄存器		
地址	00E8_060CH		
复位值	0000_0000H		

位描述

位	位名	权限	功能
31-26	RFU		-
25~0	wdt_cnt	R	Watch Dog 当前计数值, 仅可读

3.3.7 CRC 模块

3.3.7.1 CRC_CFG

寄存器	CRC_CFG		
功能	CRC 控制寄存器		
地址	0x00E8_0700		
复位值	0x0000_0000		

位描述

位	位名	权限	功能
31-4	RFU	-	RFU
3	crc_endian_byte_fix	RW	CRC 输入数据 endian 转换形式设置: crc_endian_byte_fix: 进行 word 内的 byte endian 切换; crc_endian_bit_fix :为进行 word/byte 内的 bit 颠倒操作。 原输入信号为 data_wr[31:0]情况下, 不同配置下对 data_wr:
2	crc_endian_bit_fix	RW	

			00: data_wr[31:0]不变。 01: {07-00, 15-08, 23-16, 31-24}, 进行 4byte 的 endian 颠倒。 10: {24-31, 16-23, 08-15, 00-07}, 进行 4 个 byte 内部 bit 的 endian 颠倒, 但是 4 个 byte 位置不变。 11: {00-07, 08-15, 16-23, 24-31}, 进行 word 内全部 bit 的颠倒。 注: [3:2] bit 建议配置为 2'b00。
1	crc_data_width	RW	CRC 输入数据长度, 即 crc 数据写入寄存器实际有效数据长度: 1'b0: 8 位, 即只对 crc_data_in 寄存器数据的最低 8bit 进行计算。 1'b1: 32 位, 即对 crc_data_in 寄存器所有 32 bit 数据进行计算。
0	crc_status	RW	写 1 进行 CRC 运算。 写 0 停止 CRC 计算。

3.3.7.2 CRC_DATAIN

寄存器	CRC_DATAIN
功能	CRC 数据写入寄存器
地址	0x00E8_0704
复位值	0x0000_0000

位描述

位	位名	权限	功能
31-0	crc_in	W	CRC 运算写入数据。 根据 crc_cfg.crc_datain_width 寄存器位的定义, 每次有 8bit 或者 32 bit 参与 CRC 计算。 对这个寄存器每写一次, 则 crc 计算一次。

3.3.7.3 CRC_DATA

寄存器	CRC_DATA
功能	CRC 数据寄存器
地址	0x00E8_0708
复位值	0xFFFF_FFFF

位描述

位	位名	权限	功能
31-16	RFU		

15-0	crc	RW	CRC 运算结果寄存器。 对此寄存器的写入可设置 CRC 计算器的初始值。CRC16 取其全部 16 位。 每次 CRC 需要复位的情况下只需要对此寄存器进行一次写入操作。CRC 计算器就立即变成此寄存器数据。
------	-----	----	---

3.3.8 INTERRUPT 模块

3.3.8.1 NVIC_ISER

寄存器名	NVIC_ISER
功能	中断使能配置寄存器
地址	0xE000_E100
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~0	SETENA	RW	中断使能配置寄存器，每一个位对应一个相同编号的外部中断源。高有效。 对该寄存器位的写操作定义如下： 0：无效操作 1：使能该位对应外部中断 对该寄存器位的读操作定义如下： 0：该位对应外部中断未使能 1：该位对应外部中断被使能

3.3.8.2 NVIC_ICER

寄存器名	NVIC_ICER
功能	中断使能清除寄存器
地址	0xE000_E180
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~0	CLRENA	RW	中断使能清除寄存器，每一个位对应一个相同编号的外部中断源。高有效。 对该寄存器位的写操作定义如下： 0：无效操作 1：清除该位对应外部中断使能 对该寄存器位的读操作定义如下： 0：不清除该位对应的外部中断使能 1：清除该位对应的外部中断使能

3.3.8.3 NVIC_ISPR

寄存器名	NVIC_ISPR
功能	中断等待配置寄存器



地址	0xE000_E200
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~0	SETPEND	RW	更改中断到等待状态配置寄存器，每一个位对应一个相同编号的外部中断源。高有效。 对该寄存器位的写操作定义如下： 0：无效操作 1：改变该位对应外部中断到等待状态 对该寄存器位的读操作定义如下： 0：该位对应外部中断未处于等待状态 1：该位对应外部中断处于等待状态

3.3.8.4 NVIC_ICPR

寄存器名	NVIC_ICPR
功能	中断等待清除寄存器
地址	0xE000_E280
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~0	CLRPEND	RW	清除中断的等待状态寄存器，每一个位对应一个相同编号的外部中断源。高有效。 对该寄存器位的写操作定义如下： 0：无效操作 1：清除该位对应外部中断的等待状态 对该寄存器位的读操作定义如下： 0：对应外部中断未处于等待状态 1：对应外部中断处于等待状态

3.3.8.5 INT_STATUS

寄存器名	INT_STATUS
功能	中断状态寄存器
地址	0x00E8_0804
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~18	RFU	—	—
17	wakeup_int	R	芯片从低功耗唤醒中断，高有效。只读，该中断在 PMU 模块寄存器中清零。
16	data_flash_check_fail_int	R	Flash 数据校验失败中断，高有效。只读，该中断在 flash_intf 模块寄存器中清零。
15~12	RFU	—	—

11	Timer2_int	R	TIMER2 中断, 高有效。只读, 该中断在 TIMER 模块寄存器中清零。
10	Timer1_int	R	TIMER1 中断, 高有效。只读, 该中断在 TIMER 模块寄存器中清零。
9~7	RFU	—	—
6	spi_int	R	SPI 接口中断, 高有效。只读, 该中断在 SPI 接口模块寄存器中清零。
5	RFU	—	—
4	pad_int	R	PAD 中断, 高有效。只读, 该中断在 PAD 模块寄存器中清零。
3	i2c_int	R	I2C 接口中断, 高有效。只读, 该中断在 I2C 接口模块寄存器中清零。
2	ct_int	R	ISO7816 接口中断, 高有效。只读, 该中断在 ISO7816 接口模块寄存器中清零。
1	ct_rst_n_int_flag	RDW ₁	ISO7816 接口复位信号电平状态中断标志, 高有效。硬件置位, 软件写 1 清除。 0: ISO7816 接口复位信号电平未发生变化 1: ISO7816 接口复位信号电平发生过变化 (从高电平变成低电平, 或者从低电平变成高电平)
0	ct_clk_on_off_int_flag	RDW ₁	ISO7816 接口时钟状态中断标志, 高有效。硬件置位, 软件写 1 清除。 0: ISO7816 接口时钟信号状态未发生变化 1: ISO7816 接口时钟信号状态发生过变化 (接口时钟信号从无效到有效, 或者从有效到无效)

注: 写 1 清除相关中断, 写零忽略, 不产生任何操作。

3.3.8.6 SIG_STATUS

寄存器名	SIG_STATUS
功能	信号状态寄存器
地址	0x00E8_0808
复位值	0x0000_0000

位描述

位	位名	权限	功能描述
31~2	RFU	—	—
1	ct_rst_n_syn	R	ISO7816 接口复位信号电平状态标志, 高有效。
0	ct_clk_on_syn	R	ISO7816 接口时钟状态标志, 高有效。

4 异常和中断

4.1 概述

异常和中断处理是处理器根据内部或外部的异常或中断事件从正常的程序处理转入特定的异常及中断处理程序。异常及中断处理利用异常中断向量表跳转到异常中断服务程序的入口。

芯片复位后，所有的中断信号默认是关闭的，需要用户手动开启中断使能。

- 引起异常的外部事件包括：外部设备的中断请求、读写访问错误和硬件重启
- 引起异常的内部事件包括：非法指令、非对齐错误、特权异常、SVC 请求管理调用、PendSV 可挂起的系统调用和 SysTick 定时器溢出。而且，非法指令、load 和 store 访问的地址没有对齐还有用户模式下执行特权指令都会产生异常。

向量表在处理器地址映射中，只有重启向量是固定的。

下表是 FM15L013 的向量表，其中用户需要处理的异常以及中断包括：

编号和地址				FM15L013 向量表	
类型	向量号	向量偏移 (十六进制)	中断号	向量分配	优先级
异常	15	3C		SysTick	可编程
	20	50	4	gpio_int	可编程
	26	68	10	timer1_int	可编程
	27	6C	11	timer2_int	可编程
	32	80	16	data_NVM_chk_fail_int	可编程
	33	84	17	wakeup_int	可编程
	36	88	18	soft_int	可编程

表 5-1: FM15L013 异常和中断向量表

4.2 异常

SC000 的异常包含以下几种类型：

4.2.1 复位

复位 CPU 内部寄存器，同时，VTOR（向量基准寄存器）也被清零，异常向量表的基准地址就是 0x00000000，CPU 从异常向量表中以偏移地址 0x04 为偏移地址读取异常向量，并把它装载到程序计数器（PC）。异常处理器把控制权转移到 PC 指向的地址。

4.2.2 NMI

在 SC000 内部 NMI 不能被屏蔽，芯片外部会对输入 SC000 端口的 NMI 中断信号做屏

蔽处理。当标志信号有效后，芯片会被直接复位，软件无需对中断进行处理。

4.2.3 硬件错误

硬件错误异常用于处理程序执行时产生的错误，这些错误通常是非法操作。

4.2.4 SysTick

NVIC 中的 SysTick 定时器为 OS 应用可以利用的另外一个特性。几乎所有操作系统的运行都需要上下文切换，而这一过程通常需要依靠定时器产生定时中断来完成。

4.3 中断

FM15L013 的外部中断信号都是电平中断，高电平表示中断有效，所以需要软件响应中断后，在中断服务程序中对中断标志进行清零操作，否则退出中断后会重新进入中断流程。FM15L013 支持的外部中断包括以下几种：

4.3.1 管脚电平中断

该中断的中断向量号为 4，中断向量表地址为“基地址+0x50”。

中断信号为 `gpio_int`，在芯片的外部管脚电平发生指定条件的变化或电平条件产生后，就产生管脚中断。只要定义的中断条件产生，无论这些外部管脚是作为通讯端口的特定功能管脚，或者作为 GPIO，都会产生管脚电平中断。

4.3.2 计数器 1 计数中断

该中断的中断向量号为 10，中断向量表地址为“基地址+0x68”。

中断信号为 `timer1_int`，当 TIMER1 计数模块达到设定的计数值时，会产生中断信号用来通知 CPU 计数完成的状态。

4.3.3 计数器 2 计数中断

该中断的中断向量号为 11，中断向量表地址为“基地址+0x6C”。

中断信号为 `timer2_int`，当 TIMER2 计数模块达到设定的计数值时，会产生中断信号用来通知 CPU 计数完成的状态。

4.3.4 数据 NVM 校验错误报警中断

该中断的中断向量号为 16，中断向量表地址为“基地址+0x80”。

中断信号为 `data_NVM_chk_fail_int`，当芯片读访问数据 NVM 时，读出的数据和校验位不匹配时，会产生该中断信号用来通知 CPU 当前读出的数据 NVM 中的数据校验错误。

4.3.5 软中断

该中断的中断向量号为 18，中断向量表地址为“基地址+0x88”。

中断信号为 `soft_int`，当 CPU 写 `sc000` 内部的中断使能寄存器后，会产生一个软中断。该中断信号的相关操作寄存器都在 `SC000` 内部寄存器。请查阅相关 `SC000` 的中断寄存器说明文档。

复旦微电子

5 NVM 擦写流程

5.1 概述

FM15L013 允许对 NVM 进行数据的反复擦写。NVM 擦写操作的基本概念：

分成两个独立的步骤：

- 先进行擦操作（Erase）
- 接着进行编程操作（Program）

擦操作以 page 或 sector 为基本单位。擦操作会将 page/sector 内的数据全部擦成“1”且只允许数据变成“1”。

编程操作以 Word 为基本单位，会将 word 数据编程成指定的 32 位数据。编程操作只允许数据从“1”变成“0”，并且允许数据维持“1”的状态。

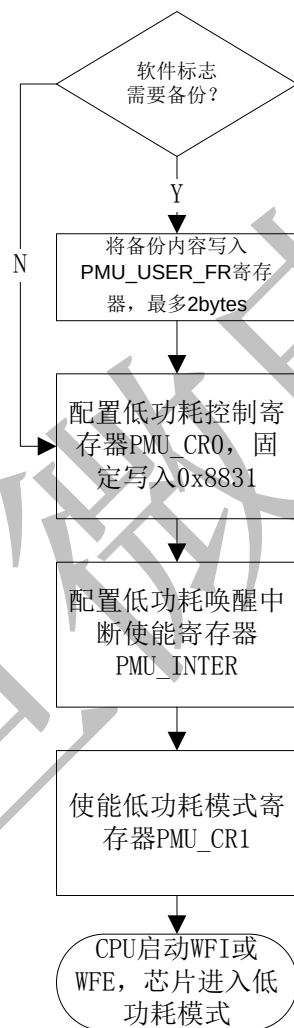
NVM 的擦和写操作均由 CPU 启动，CPU 配置 NVM 擦写流程寄存器来控制擦写。

6 低功耗流程

6.1 概述

支持两种低功耗模式，一种是正常 sleep 模式，另一种是超低功耗 PowerDown 模式。

6.2 操作流程

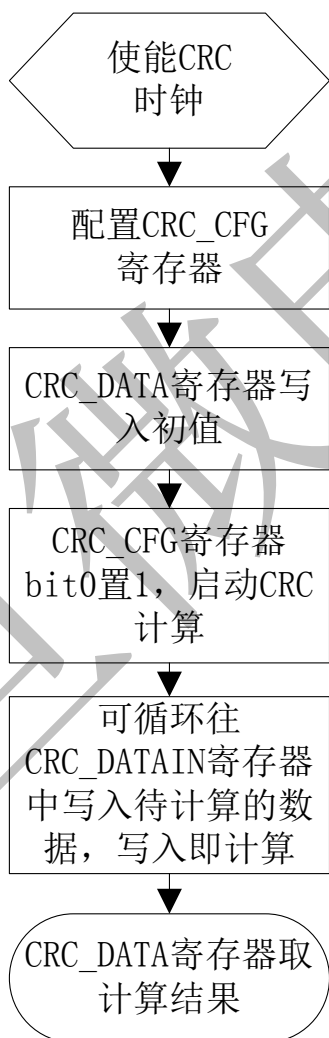


7 CRC 计算

7.1 概述

CRC 模块的功能是对数据进行 CRC 校验值的计算。CRC 的初值可配置，硬件 CRC 算法支持支持 CCITT 标准的 CRC16 计算。多项式为 $x^{16} + x^{12} + x^5 + 1$ 。

7.2 操作流程



8 时钟控制

8.1 概述

芯片的主要工作时钟 `system_clk` 是由内部环振产生，环振的频率固定为 48Mhz，用户可以通过对寄存器的配置来设置时钟分频数，从而产生系统时钟 `system_clk` 的工作频率。在芯片处于固定工作频率工作时，环振频率的配置值是由软件写寄存器来操作的。

工作时钟的频率和分频配置值的对应关系可表示为：

$$f_{system_clk} = 2 * f_{mcg_clk} / N \quad (N \text{ 通过 } func_n_cfg \text{ 寄存器配置})$$

`fmcg_clk` 为 48Mhz 和其 3/2 分频后的 32Mhz 的其中一个，选择信号由寄存器配置。

8.2 stretch 的使用

无论 CPU 是否停止，如果改变系统频率，都需注意配合 stretch 的配置进行。

芯片对外部慢速存储器 NVM 时，根据当前工作频率设置不同的 stretch 参数，可以增加对 NVM 的访问周期数来满足对 NVM 的访问时间要求。

NVM 的 stretch 分为 4 档，分别对应 1~4 个时钟访问周期。stretch 的档位配置由寄存器 (`falsh_intf_cfg` 的 `stretch_value` 寄存器位) 设定。

需要注意的是，如果需要提升频率，可能需要增加读取 NVM 的 stretch，则应在修改系统时钟频率前，修改 stretch 寄存器以增加 stretch 数值；若需要降频，则应该在修改频率寄存器降低频率后（最好等一定的 cycle 数以使系统时钟频率稳定降低），再修改 stretch 寄存器减少 stretch 数值。

9 计时器

9.1 概述

芯片支持多个计时器模块：

- CPU SC000模块内部24-Bits的SysTick定时器
- 一个16-Bits的计时器TIMER1和一个32-bits的计时器TIMER2

用户可以根据应用需求选择合适的计时器进行操作。

9.2 24-Bits 定时器

9.2.1 简介

24-Bits 的定时器 SysTick 是由 CPU SC000 模块内部实现。定时时间到达后会产生一个异常信号通知 CPU 处理。

SysTick 的设计结构是向下计数，启动定时器后，硬件自动加载软件定义的初值，在每个定时时钟的上升沿，定时器的计数减 1，当减至 0 后，会产生异常编号为 15 的 SysTick 异常，同时定时器重新装载初值，启动下一次的计数操作。

SysTick 固定采用溢出重载功能，即启动计数后定时器即循环工作。除非程序主动关闭定时器使能位。

SysTick 定时器的时钟源支持不同的配置：

- CPU内核时钟
- 外部时钟
 - a) 固定4MHz时钟
 - b) 系统时钟4/8/16/32分频
 - c) 接触接口时钟八分频
 - d) 接触接口波特率时钟

外部时钟配置详见 RCC 模块 PERICK_CFG1 寄存器。

9.2.2 寄存器定义

SysTick 由 CPU 内部的 4 个寄存器进行控制。

位	定义	访问 权限	复位值	描述
SysTick_ctrl (0xE000_E010)				
31~17	RFU	—	0	—
16	count_flag	RD	0	当定时器计数到 0 时，该位变 1，读操作后该寄存器会被清零

15~3	RFU	—	0	—
2	Clk_source	WR	0	定时器时钟源选择 1: CPU 时钟 0: 不支持
1	tick_int	RW	0	定时器中断使能, 高有效。当该位置位后, 定时器计数到 0 时会产生异常
0	enable	RW	0	使能位, 高有效。置 1 时定时器使能操作。
SysTick_load (0xE000_E014)				
31~24	RFU	—	0	—
23~0	reload	RW	24'hFF_FFFF	定时器的重装载值
SysTick_value (0xE000_E018)				
31~24	RFU	—	0	—
23~0	Value	RW	24'hFF_FFFF	读操作读出定时器的当前数值, 写入任何值都会清除寄存器, 同时也清除 count_flag 标志。
SysTick_calib (0xE000_E01C)				
31	Noref	R	0	支持定时时钟选择, 通过 clk_source 位选择定时时钟源
30	Skey	R	0	TENMS 域准确
29~24	RFU	—	0	—
23~0	Ten_ms	R	24'h01_3880	10ms 的校准值

表 9-1: SysTick 定时器的寄存器列表

9.2.3 操作流程

SysTick 定时器支持通过轮询或中断的方式操作。

使用轮询的方式, 程序可以读取 SysTick_ctrl 控制寄存器的 count_flag 位来进行判断, 如果该标志置位, 表明定时器已经减到 0。

一个典型的 SysTic 定时器的操作流程如下所示:

- 1) 关闭SysTick定时器
- 2) 配置SysTick定时器的重装载值reload
- 3) 向SysTick定时器的计数寄存器写0清除当前计数值
- 4) 启动SysTick定时器开始计数
- 5) 读取count_flag标志并进行判断, 如果该标志为1, 代表定时时间到达。

9.3 独立计时器

9.3.1 概述

芯片内部共支持两个 16Bits 的计时器模块: TIMER1 和 TIMER2, 它们产生不同的中断信号, 计时器的设计基本一样。

启动 Timer 后, 计时器会将寄存器 `TIMERx_COUNTERy` 寄存器清零, 然后在每个输入时钟的上升沿, 硬件都会对计数寄存器 `TIMERx_COUNTERy` 进行加 1 操作, 当计数寄存器 `TIMERx_COUNTERy` 的值与 `TIMERx_PRESETy` 的预置值一样时, 计数值达到, 产生 TIMER 的中断信号。

计时器支持自动重载功能, 允许计时器进行循环计数操作。

计时器的停止方式:

- 计数值达到后硬件自动停止, 同时会产生计数到达的中断信号
- 软件写寄存器主动停止计时器, 此时可以读取count寄存器值获取当前的计数值

计时器的时钟源支持不同的配置:

- 系统时钟时钟的4/8/16/32分频
- 接触接口接口时钟的8分频
- 接触接口波特率时钟

9.3.2 计时器精度与计时范围

按照计时器的工作模式和输入时钟的配置不同, 计时器的精度与计时范围都各不相同, 本章节描述的计时精度定义为工作时钟周期乘以最小计时周期单位, 计时范围定义为计时精度乘以计时器的计时范围。

计时器计数范围为 $1 \sim 2^{16}-1$, 计时精度为 T_{clk} 。下表描述了 16 位计时器的精度与范围:

计数时钟	典型值	计数精度(us)	计数最大值(ms)
接触接口输入时钟 8 分频	1M	8	524.3
	3.58M	2.23	146.1
	5M	1.6	104.9
接触接口波特率时钟	372 分频@3.58M	103.8	6827
	31 分频@3.58M	8.65	569
系统时钟 4 分频	1M	4	262.1
	32M	0.125	8.2
系统时钟 8 分频	1M	8	524.3
	32M	0.25	16.4
系统时钟 16 分频	1M	16	1048.6
	32M	0.5	32.8
系统时钟 32 分频	1M	32	2097.2
	32M	1	65.5

表 10-1: 16 位计时器的精度与范围

9.3.3 操作流程

使用 TIMER 进行计数操作的通用操作流程如下所示:

1. RCC模块PERICKL_CFG0寄存器中开启timer模块时钟, PERICKL_CFG1寄存器中配置timer计数时钟

2. 写配置寄存器，完成TIMER计数器操作的相关配置
 - a) 写TIMERx_CONFIG寄存器，完成TIMER模式，中断和DMA请求的各种配置。
 - b) 写TIMERx_PRESETy寄存器，完成目标计数值的配置
3. 写TIMERx_CTRLy寄存器的START位，启动TIMER计时操作功能
4. 等待TIMER计时达到preset的预置值后会产生timer的溢出中断，CPU进入timer中断服务程序中进行处理。
5. 如果timer被配置成重载功能使能后，timer会循环计数，在产生溢出中断的同时，TIMER会将当前的计数值清零后重新启动新的计数功能，如果CPU需要主动停止循环计数的timer，需要清除TIMERx_CTRLy寄存器的START位，主动停止timer计数。
6. TIMER溢出中断未产生前，如果CPU需要主动停止timer计时操作，也可以主动清除TIMERx_CTRLy寄存器的START位，停止timer计数。此时可以读取TIMER_COUNTER寄存器获取已经计时的周期。

10 WDT

芯片支持一个硬件看门狗电路。硬件 WDT 模块是一个 26 位的计数器，每个 CPU 时钟周期计时器加 1，在达到配置的计时时间后硬件会自动产生一个复位信号，复位整个芯片。在程序执行异常情况下提供了一种恢复正常流程的方法。

复位后 WDT 模块默认不工作，应该由 CPU 启动，启动后不可以停止。WDT 的溢出时间可以由用户写 WDT_TIMER_CONFIG 寄存器进行配置，但是只能在启动 WDT 之前进行配置，WDT 启动后溢出时间就不可配置。

在启动 WDT 计时后，用户通过向 WDT_CLR 寄存器写特定的数据可以清零 WDT 计时器，并启动 WDT 开始运行。如果在溢出时间之内用户没有做清零 WDT 计时器的操作，WDT 就会产生硬件复位信号，复位整个芯片。



版本信息

版本号	发布日期	页数	章节或图表	更改说明
0.1	2021.11	43		Preliminary 发布

复旦微电子



上海复旦微电子集团股份有限公司销售及服务中心

上海复旦微电子集团股份有限公司

地址：上海市国泰路 127 号 4 号楼

邮编：200433

电话：(86-021) 6565 5050

传真：(86-021) 6565 9115

上海复旦微电子（香港）有限公司

地址：香港九龙尖沙咀东嘉连威老道 98 号东海商业中心 5 楼 506 室

电话：(852) 2116 3288 2116 3338

传真：(852) 2116 0882

北京办事处

地址：北京市东城区东直门北小街青龙胡同 1 号歌华大厦 B 座 423 室

邮编：100007

电话：(86-10) 8418 6608

传真：(86-10) 8418 6211

深圳办事处

地址：深圳南山区西丽街道留仙大道创智云城 A7 座 2306-08

邮编：518000

电话：(86-0755) 8335 0911 8335 1011 8335 2011 8335 0611

传真：(86-0755) 8335 9011

台湾办事处

地址：台北市 114 内湖区内湖路一段 252 号 12 楼 1225 室

电话：(886-2) 7721 1889

传真：(886-2) 7722 3888

新加坡办事处

地址：47 Kallang Pudding Road, #08-06, The Crescent @ Kallang, Singapore 349318

电话：(65) 6443 0860

传真：(65) 6443 1215

复旦微电子（美国）公司

地址：97 E Brokaw Road, Suite 320, San Jose, CA 95112

电话：(+1)408-335-6936

公司网址：<http://www.fmsh.com/>